

Keehan_Alyssa_HW3

Name: Alyssa Keehan

GitHub Username: akeehan56

GitHub Links: [Class Repo](#) | [HW3](#)

Email: akeehan@usc.edu

USC ID: 3429542433

1. Time Series Classification Part 1: Feature Creation/Extraction

(a) Download Data

Package imports

```
In [1]: from pathlib import Path
import re
import numpy as np
import pandas as pd
```

Get the AReM Data Set

```
In [2]: data_dir = Path("../data/AReM")

activities = [
    "bending1",
    "bending2",
    "cycling",
    "lying",
    "sitting",
    "standing",
    "walking"
]

columns = [
    "time",
    "avg_rss12",
    "var_rss12",
    "avg_rss13",
    "var_rss13",
    "avg_rss23",
```

```
"var_rss23"
]
```

(b) Test and Train Data

In [3]: *# split each of the data sets into*

```
test_dataset_numbers = {
    "bending1": [1, 2],
    "bending2": [1, 2],
    "cycling": [1, 2, 3],
    "lying": [1, 2, 3],
    "sitting": [1, 2, 3],
    "standing": [1, 2, 3],
    "walking": [1, 2, 3]
}
```

In [4]: `train_instances = []`
`test_instances = []`

```
for activity in activities:
    folder_path = data_dir / activity

    # Find and sort all csv files in the activity folder
    files = sorted(
        folder_path.glob("dataset*.csv"),
        key=lambda file: int(file.stem.replace("dataset", ""))
    )

    for file_path in files:
        dataset_num = int(file_path.stem.replace("dataset", ""))

        # only the first seven columns are retained
        df = pd.read_csv(
            file_path,
            skiprows=5,
            header=None,
            names=columns,
            sep=r"[,\s]+",
            engine="python",
            usecols=range(7)
        )

        # ensure that all values are numeric
        df = df.apply(pd.to_numeric, errors="raise")

        instance = {
            "activity": activity,
            "dataset": dataset_num,
            "file": file_path.name,
            "data": df
        }

        if dataset_num in test_dataset_numbers[activity]:
            test_instances.append(instance)
```

```
else:  
    train_instances.append(instance)
```

```
In [5]: print("Training instances:", len(train_instances))  
        print("Testing instances:", len(test_instances))  
        print("Total instances:", len(train_instances) + len(test_instances))
```

Training instances: 69

Testing instances: 19

Total instances: 88

Answer: Datasets 1 and 2 from bending1 and bending2 were assigned to the test set. Datasets 1, 2, and 3 from each of the other activity folders were also assigned to the test set. All remaining datasets were assigned to the training set. This produced 69 training instances and 19 test instances.

(c) Feature Extraction

i. Research

Common time-domain features used for time-series classification include:

- **Measures of central tendency:** mean and median
- **Measures of variability:** variance and standard deviation
- **Measures of distribution shape:** skewness and kurtosis.
- **Measures of boundaries:** minimum, maximum, range and interquartile range (IQR)
- **Signal magnitude features:** root mean square, absolute energy, and mean absolute value.
- **Change-based features:** mean absolute change, maximum change, and the number of zero or mean crossings.
- **Temporal-dependence features:** autocorrelation and partial autocorrelation.
- **Pattern-based features:** number of peaks, peak locations, and the lengths of consecutive values above or below the mean.
- **Trend features:** slope, intercept, and other linear-trend measurements.

These features convert each time series into a fixed set of numerical values that can be used by standard classification algorithms.

Links to Sources:

- TSFEL. (n.d.). *List of available features*.
https://tsfel.readthedocs.io/en/latest/descriptions/feature_list.html
- tsfresh. (n.d.). *Overview on extracted features*.
https://tsfresh.readthedocs.io/en/latest/text/list_of_features.html

ii. Extraction

```
In [6]: # give each activity an order number for sorting
activity_order = {
    activity: index
    for index, activity in enumerate(activities)
}

# combine the training and testing instances and sort them by activity and dataset
all_instances = sorted(
    train_instances + test_instances,
    key=lambda instance: (
        activity_order[instance["activity"]],
        instance["dataset"]
    )
)

print("Number of instances:", len(all_instances))
```

Number of instances: 88

```
In [7]: time_series_columns = [
    "avg_rss12",
    "var_rss12",
    "avg_rss13",
    "var_rss13",
    "avg_rss23",
    "var_rss23"
]

# holds the extracted features for each instance
feature_rows = []
# holds the names of the extracted features
feature_columns = []

for instance_number, instance in enumerate(all_instances, start=1):
    row = {
        "Instance": instance_number,
        "Activity": instance["activity"],
        "Dataset": instance["dataset"]
    }

    if instance["dataset"] in test_dataset_numbers[instance["activity"]]:
        row["Split"] = "test"
    else:
        row["Split"] = "train"

    for series_number, column in enumerate(time_series_columns, start=1):
        values = instance["data"][column]

        row[f"min{series_number}"] = values.min()
        row[f"max{series_number}"] = values.max()
        row[f"mean{series_number}"] = values.mean()
        row[f"median{series_number}"] = values.median()
        row[f"std{series_number}"] = values.std(ddof=1)
        row[f"1stQuart{series_number}"] = values.quantile(0.25)
        row[f"3rdQuart{series_number}"] = values.quantile(0.75)
```

```

        if instance_number == 1:
            feature_columns.extend([
                f"min{series_number}",
                f"max{series_number}",
                f"mean{series_number}",
                f"median{series_number}",
                f"std{series_number}",
                f"1stQuart{series_number}",
                f"3rdQuart{series_number}"
            ])

        feature_rows.append(row)

feature_dataset = pd.DataFrame(feature_rows)

feature_dataset = feature_dataset[
    ["Instance", "Activity", "Dataset", "Split"] + feature_columns
]

feature_dataset

```

Out[7]:

	Instance	Activity	Dataset	Split	min1	max1	mean1	median1	std1
0	1	bending1	1	test	37.25	45.00	40.624792	40.50	1.476967
1	2	bending1	2	test	38.00	45.67	42.812812	42.50	1.435550
2	3	bending1	3	train	35.00	47.40	43.954500	44.33	1.558835
3	4	bending1	4	train	33.00	47.75	42.179812	43.50	3.670666
4	5	bending1	5	train	33.00	45.75	41.678063	41.75	2.243490
...	...	...	...	...	...	...	...	...	...
83	84	walking	11	train	19.50	45.33	33.586875	34.25	4.650935
84	85	walking	12	train	19.75	45.50	34.322750	35.25	4.752477
85	86	walking	13	train	19.50	46.00	34.546229	35.25	4.842294
86	87	walking	14	train	23.50	46.25	34.873229	35.25	4.531720
87	88	walking	15	train	19.25	44.00	34.473188	35.00	4.796705

88 rows × 46 columns

```

In [8]: print("Dataset shape:", feature_dataset.shape)
        print("Number of extracted features:", len(feature_columns))

        print(
            "Missing feature values:",
            feature_dataset[feature_columns].isna().sum().sum()
        )

```

Dataset shape: (88, 46)
 Number of extracted features: 42
 Missing feature values: 0

iii. Standard Deviation

```
In [9]: feature_values = feature_dataset[feature_columns]

std_estimates = feature_values.std(axis=0, ddof=1)

std_estimates
```

```
Out[9]: min1          9.569975
max1          4.394362
mean1         5.335718
median1       5.440054
std1          1.772153
1stQuart1     6.153590
3rdQuart1     5.138925
min2          0.000000
max2          5.062729
mean2         1.574164
median2       1.412244
std2          0.884105
1stQuart2     0.946386
3rdQuart2     2.125266
min3          2.956462
max3          4.875137
mean3         4.008380
median3       4.036396
std3          0.946710
1stQuart3     4.220658
3rdQuart3     4.171628
min4          0.000000
max4          2.183625
mean4         1.166114
median4       1.145586
std4          0.458242
1stQuart4     0.843620
3rdQuart4     1.552504
min5          6.124001
max5          5.741238
mean5         5.675593
median5       5.813782
std5          1.024898
1stQuart5     6.096465
3rdQuart5     5.531720
min6          0.045838
max6          2.518921
mean6         1.154812
median6       1.086474
std6          0.517617
1stQuart6     0.758584
3rdQuart6     1.523599
dtype: float64
```

```
In [10]: rng = np.random.default_rng(42)

n_bootstrap = 10000
```

```
n_instances = len(feature_values)
n_features = len(feature_columns)

bootstrap_std = np.empty((n_bootstrap, n_features))

feature_array = feature_values.to_numpy()

# randomly sample instances with replacement and compute the standard deviation
for bootstrap_number in range(n_bootstrap):
    sampled_indices = rng.integers(
        low=0,
        high=n_instances,
        size=n_instances
    )

    bootstrap_sample = feature_array[sampled_indices]

    bootstrap_std[bootstrap_number] = np.std(
        bootstrap_sample,
        axis=0,
        ddof=1
    )

ci_lower = np.percentile(bootstrap_std, 5, axis=0)
ci_upper = np.percentile(bootstrap_std, 95, axis=0)

bootstrap_results = pd.DataFrame({
    "Feature": feature_columns,
    "Standard Deviation": std_estimates.values,
    "90% CI Lower": ci_lower,
    "90% CI Upper": ci_upper
})

bootstrap_results.round(3)
```

Out[10]:

	Feature	Standard Deviation	90% CI Lower	90% CI Upper
0	min1	9.570	8.236	10.774
1	max1	4.394	3.340	5.293
2	mean1	5.336	4.704	5.876
3	median1	5.440	4.784	6.006
4	std1	1.772	1.569	1.950
5	1stQuart1	6.154	5.574	6.649
6	3rdQuart1	5.139	4.339	5.834
7	min2	0.000	0.000	0.000
8	max2	5.063	4.631	5.401
9	mean2	1.574	1.396	1.706
10	median2	1.412	1.238	1.544
11	std2	0.884	0.801	0.943
12	1stQuart2	0.946	0.829	1.035
13	3rdQuart2	2.125	1.893	2.297
14	min3	2.956	2.756	3.112
15	max3	4.875	4.155	5.470
16	mean3	4.008	3.407	4.499
17	median3	4.036	3.422	4.545
18	std3	0.947	0.763	1.123
19	1stQuart3	4.221	3.626	4.709
20	3rdQuart3	4.172	3.540	4.699
21	min4	0.000	0.000	0.000
22	max4	2.184	1.971	2.363
23	mean4	1.166	1.075	1.222
24	median4	1.146	1.054	1.201
25	std4	0.458	0.421	0.486
26	1stQuart4	0.844	0.773	0.891
27	3rdQuart4	1.553	1.432	1.627
28	min5	6.124	4.438	7.505
29	max5	5.741	4.744	6.560
30	mean5	5.676	4.436	6.718
31	median5	5.814	4.522	6.899

	Feature	Standard Deviation	90% CI Lower	90% CI Upper
32	std5	1.025	0.816	1.218
33	1stQuart5	6.096	4.802	7.188
34	3rdQuart5	5.532	4.358	6.514
35	min6	0.046	0.000	0.078
36	max6	2.519	2.253	2.763
37	mean6	1.155	1.062	1.215
38	median6	1.086	0.994	1.148
39	std6	0.518	0.479	0.545
40	1stQuart6	0.759	0.689	0.808
41	3rdQuart6	1.524	1.401	1.600

Answer:

I calculated the standard deviation of each of the 42 features across all 88 instances. I then created 10,000 bootstrap samples by resampling the instances with replacement and recalculated the standard deviations for each sample. The 5th and 95th percentiles of the bootstrap results were used as the lower and upper bounds of the 90% confidence intervals.

iv. Select Features

```
In [11]: selected_features = [
          column
          for column in feature_columns
          if column.startswith(("min", "mean", "max"))
        ]

selected_features
```

```
Out[11]: ['min1',
          'max1',
          'mean1',
          'min2',
          'max2',
          'mean2',
          'min3',
          'max3',
          'mean3',
          'min4',
          'max4',
          'mean4',
          'min5',
          'max5',
          'mean5',
          'min6',
          'max6',
          'mean6']
```

Answer:

I selected the minimum, mean, and maximum as the three most important time-domain features. The mean represents the overall level of a time series, while the minimum and maximum capture its lower and upper extremes. Together, they summarize the location and range of the signal and may help distinguish activity patterns with different RSS levels.

2. ISLR 3.7.4

I collect a set of data ($n = 100$ observations) containing a single predictor and a quantitative response. I then fit a linear regression model to the data, as well as a separate cubic regression, i.e. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \varepsilon$

(a) Linear Train

Suppose that the true relationship between X and Y is linear, i.e. $Y = \beta_0 + \beta_1 X + \varepsilon$. Consider the training residual sum of squares (RSS) for the linear regression, and also the training RSS for the cubic regression. Would we expect one to be lower than the other, would we expect them to be the same, or is there not enough information to tell? Justify your answer.

Answer: We expect the cubic regression training RSS to be less than or equal to the linear regression training RSS .

The cubic model contains the linear model as a special case when $(\beta_2 = \beta_3 = 0)$. Its additional terms give it more flexibility to fit the training observations, including some random noise. Therefore, adding X^2 and X^3 cannot increase the training RSS , even though the true relationship is linear.

(b) Linear Test

Answer (a) using test rather than training RSS .

Answer: We expect the linear regression to have a lower test RSS .

Because the true relationship is linear, the X^2 and X^3 terms in the cubic model are unnecessary. Although these additional terms may help the cubic model fit the training data, they may also cause it to overfit random noise that does not appear in new data. This additional flexibility increases the model's variance. Because the linear model matches the true relationship and is simpler, we expect it to generalize better and produce a lower test RSS .

(c) Not Linear Train

Suppose that the true relationship between X and Y is not linear, but we don't know how far it is from linear. Consider the training RSS for the linear regression, and also the training RSS for the cubic regression. Would we expect one to be lower than the other, would we expect them to be the same, or is there not enough information to tell? Justify your answer.

Answer: We again expect the cubic regression to have a training RSS that is less than or equal to the linear regression's training RSS .

The conclusion is still the same as in part a, since the cubic regression model is more flexible it can always fit to training data as least as good as the linear model.

(d) Not Linear Testing

Answer (c) using test rather than training RSS .

Answer: There is not enough information to tell which model will have lower test RSS.

The result depends on the form and degree of the nonlinearity:

- If the true relationship is well approximated by a cubic function, the cubic regression may have lower test RSS because its reduced bias outweighs its increased variance.
- If the relationship is only slightly nonlinear, the linear model may perform better because the cubic model's additional variance could outweigh its reduction in bias.
- If the relationship has a form that a cubic model cannot approximate well, either model could perform better.

Therefore, without knowing how nonlinear the true relationship is, we cannot determine which model will have the lower test RSS.

3. ISLR 3.7.3 - Extra Practice

Suppose we have a data set with five predictors:

- $X_1 = \text{GPA}$
- $X_2 = \text{IQ}$
- $X_3 = \text{Level}$, where 1 represents College and 0 represents High School
- $X_4 = \text{Interaction between GPA and IQ}$
- $X_5 = \text{Interaction between GPA and Level}$

The response variable is starting salary after graduation, measured in thousands of dollars.

Suppose we use least squares to fit the model and obtain the following coefficient estimates:

$$\hat{\beta}_0 = 50, \quad \hat{\beta}_1 = 20, \quad \hat{\beta}_2 = 0.07, \quad \hat{\beta}_3 = 35, \quad \hat{\beta}_4 = 0.01, \quad \hat{\beta}_5 = -10.$$

$$\widehat{\text{Salary}} = 50 + 20X_1 + 0.07X_2 + 35X_3 + 0.01X_4 - 10X_5$$

where

$$X_4 = \text{GPA} \times \text{IQ} \text{ and } X_5 = \text{GPA} \times \text{Level}.$$

Substituting the variable definitions gives

$$\widehat{\text{Salary}} = 50 + 20(\text{GPA}) + 0.07(\text{IQ}) + 35(\text{Level}) + 0.01(\text{GPA} \times \text{IQ}) - 10(\text{GPA} \times \text{L})$$

(a) Which answer is correct, and why?

1. For a fixed value of IQ and GPA, high school graduates earn more, on average, than college graduates.

2. For a fixed value of IQ and GPA, college graduates earn more, on average, than high school graduates.
3. For a fixed value of IQ and GPA, high school graduates earn more, on average, than college graduates, provided that GPA is high enough.
4. For a fixed value of IQ and GPA, college graduates earn more, on average, than high school graduates, provided that GPA is high enough.

Answer: #3 -> For fixed IQ and GPA, high school graduates earn more, on average, than college graduates, provided that GPA is high enough.

$$\widehat{\text{Salary}}_{\text{College}} - \widehat{\text{Salary}}_{\text{High School}} = 35 - 10(\text{GPA})$$

College graduates earn more when

- $35 - 10(\text{GPA}) > 0 \rightarrow \text{GPA} < 3.5$

High school graduates earn more when

- $35 - 10(\text{GPA}) < 0 \rightarrow \text{GPA} > 3.5$

If we're provided that the GPA is high enough, then the higher the GPA, the more the high school student earns, option 3 is the correct choice.

(b) Predict the salary of a college graduate with an IQ of 110 and a GPA of 4.0.

```
In [12]: IQ = 110
        GPA = 4.0
        Level = 1

        Salary = (50 + 20*GPA + 0.07*IQ + 35*Level + 0.01*(GPA*IQ) - 10*GPA)*1000
        print('The Predicted Salary for a college student with an IQ of 110 and a GPA of 4.0 is $', Salary)
```

The Predicted Salary for a college student with an IQ of 110 and a GPA of 4.0 is \$ 137100.0

(c) True or false: Since the coefficient for the GPA–IQ interaction term is very small, there is very little evidence of an interaction effect. Justify your answer.

Answer: False.

We cannot conclude that there is little evidence of an interaction effect solely because the coefficient is small. The size of a coefficient depends on the units and scales of the variables. For example, with a GPA of 4.0 and an IQ of 110, the interaction term contributes

$$0.01(4)(110) = 4.4$$

which represents \$4,400 additional salary. Therefore, the interaction may still have a meaningful effect.

Additionally, determining whether there is statistical evidence of an interaction requires proper hypothesis testing with a p-value or t-statistic, not just the size of the coefficient.

3. ISLR 3.7.5 - Extra Practice

Consider the fitted values that result from performing linear regression without an intercept. In this setting, the i th fitted value takes the form

$$\hat{y}_i = x_i \hat{\beta},$$

where

$$\hat{\beta} = \frac{\sum_{i'=1}^n x_{i'} y_{i'}}{\sum_{i'=1}^n x_{i'}^2}$$

Show that we can write

$$\hat{y}_i = \sum_{i'=1}^n a_{i'} y_{i'}.$$

What is $a_{i'}$?

Note: We interpret this result by saying that the fitted values from linear regression are linear combinations of the response values.

Answer:

Substitute $\hat{\beta}$ into the fitted-value equation:

$$\begin{aligned} \hat{y}_i &= x_i \left(\frac{\sum_{j=1}^n x_j y_j}{\sum_{j=1}^n x_j^2} \right) \\ &= \sum_{j=1}^n \left(\frac{x_i x_j}{\sum_{k=1}^n x_k^2} \right) y_j. \end{aligned}$$

So

$$a_j = \frac{x_i x_j}{\sum_{k=1}^n x_k^2}.$$